Solution: Search Suggestions System

Problem Statement

Given a list of distinct strings `products` and a string `searchWord`.

Determine a set of `product suggestions` after each character of the search word is typed. Every time a character is typed, return a `list` containing up to three product names from the `products` list that have the same prefix as the typed string.

If there are more than 3 matching products, return 3 lexicographically smallest products. These product names should be returned in lexicographical (alphabetical) order.

Examples

1. **Example 1**:
    - **Input**: Products: ["apple", "apricot", "application"], searchWord: "app"
    - **Expected Output**: [["apple", "apricot", "application"], ["apple", "apricot", "application"], ["apple", "application"]]
    - **Justification**: For the perfix 'a', "apple", "apricot", and "application" match. For the prefix 'ap', "apple", "apricot", and "application" match. For the prefix 'app', "apple", and "application" match
2. **Example 2**:
    - **Input**: Products: ["king", "kingdom", "kit"], searchWord: "ki"
    - **Expected Output**: [["king", "kingdom", "kit"], ["king", "kingdom", "kit"]]
    - **Justification**: All products starting with "k" are "king", "kingdom", and "kit". The list remains the same for the 'ki' prefix.
3. **Example 3**:
    - **Input**: Products: ["fantasy", "fast", "festival"], searchWord: "farm"
    - **Expected Output**: [["fantasy", "fast", "festival"], ["fantasy", "fast"], [], []]
    - **Justification**: Initially, "fantasy", "fast", and "festival" match 'f'. Moving to 'fa', only "fantasy" and "fast" match. No product matches with "far", and "farm".

Solution

To solve this problem, We will use the trie data structure to store the list of products. The trie is built by inserting each product, where each node represents a character. This structure allows us to efficiently find products that share a common prefix.

After building the trie, we process the search word by checking each of its prefixes. For each prefix, we perform a depth-first search (DFS) starting from the node matching the end of the prefix. The DFS is designed to find up to three lexicographically up to 3 smallest words that start with the given prefix. This approach of using a trie combined with DFS for each prefix ensures that we can quickly and effectively generate the required list of product suggestions.

Step-by-Step Algorithm

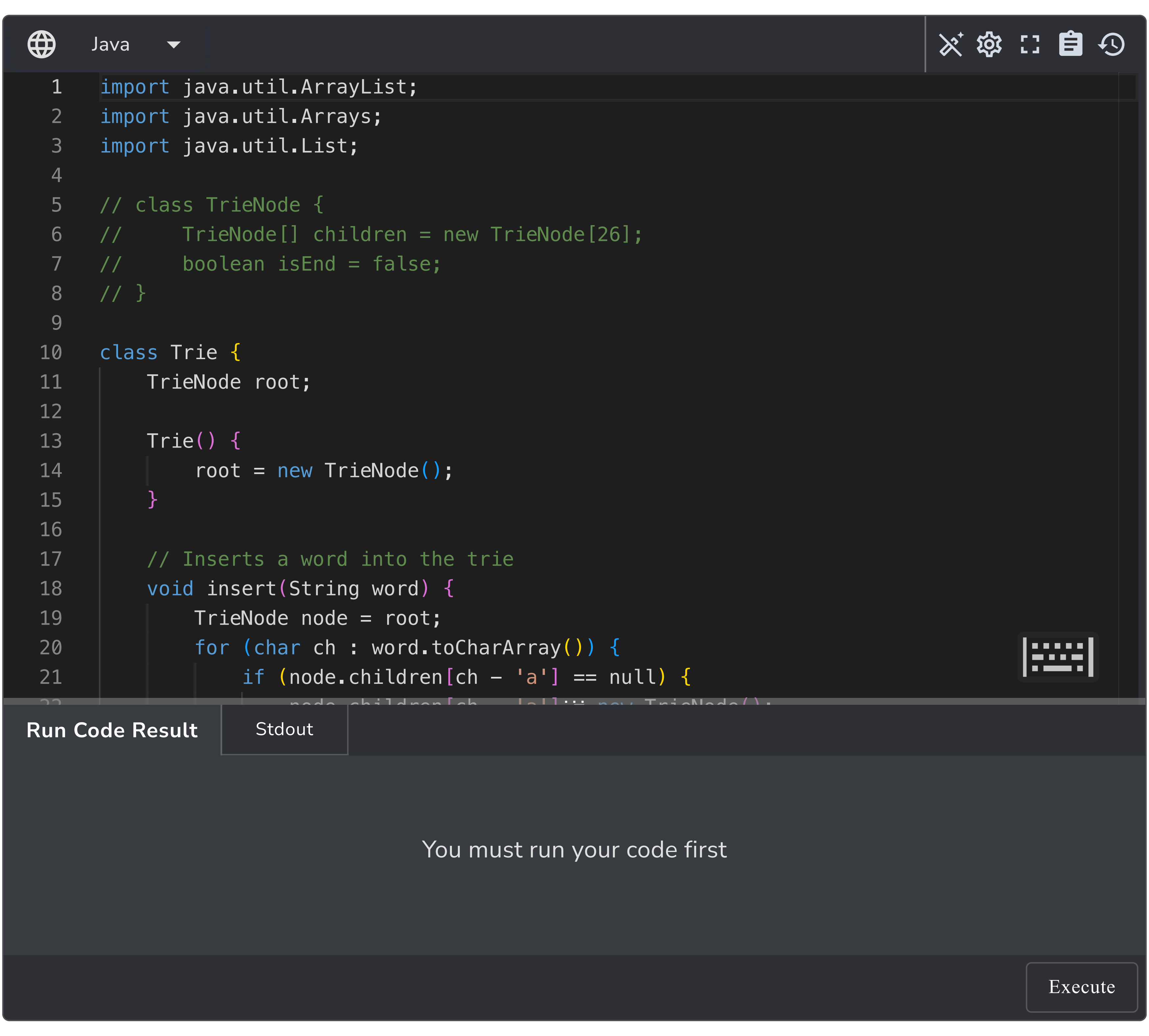
1. **Build the Trie**:
    - Create a root node representing the starting point of the trie.
    - For each product:
        - Start from the root and for each character in the product, navigate to the corresponding child node. Create a new node if it doesn't exist.
        - Mark the node corresponding to the last character of the product as a word end.
2. **Process Each Prefix of the Search Word**:
    - Initialize an empty list to store the final suggestions for each prefix.
    - For each character in the search word, form a prefix.
        - Start from the root of the trie and navigate to the node corresponding to the last character of the current prefix.
        - If the node for the current prefix doesn't exist, add an empty list to the suggestions and move to the next prefix.
3. **DFS for Each Prefix**:
    - Upon reaching the node corresponding to the current prefix, perform a DFS.
        - Initialize an empty buffer to store up to three products.
        - Explore all possible paths from the current node. If a path leads to a node marked as a word end, add the corresponding product to the buffer.
        - Stop the DFS when you have collected three products or explored all paths.
4. **Compile Suggestions**:
    - Add the buffer containing up to three products to the list of suggestions for the current prefix.
    - Continue the process for the next prefix.

Algorithm Walkthrough for Example 3

- **Input**: Products = ["fantasy", "fast", "festival"], Search Word = "farm"
- **Walkthrough**:
    1. **Building the Trie**:
        - Insert "fantasy", "fast", "festival" into the trie, creating nodes for each character.
        - Mark the end of each word in the trie.
    2. **Processing Prefixes**:
        - Prefix "f": Node exists. Proceed to DFS.
        - Prefix "fa": Node exists. Proceed to DFS.
        - Prefix "far": Node does not exist. Add an empty list to suggestions and skip DFS.
        - Prefix "farm": Node does not exist. Add an empty list to suggestions and skip DFS.
    3. **DFS for "f" and "fa"**:
        - For "f": DFS finds "fantasy", "fast", "festival". Add these to suggestions.
        - For "fa": DFS finds "fantasy", "fast". Add these to suggestions.
    4. **Final Output**:
        - Suggestions: [["fantasy", "fast", "festival"], ["fantasy", "fast"], [], []].

This walkthrough illustrates how the trie efficiently organizes the products, and the DFS ensures that only the top three lexicographical matches for each prefix are selected.

Code



```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

// class TrieNode {
//     TrieNode[] children = new TrieNode[26];
//     boolean isEnd = false;
// }

class Trie {
    TrieNode root;

    Trie() {
        root = new TrieNode();
    }

    // Inserts a word into the trie
    void insert(String word) {
        TrieNode node = root;
        for (char ch : word.toCharArray()) {
            if (node.children[ch - 'a'] == null) {
```

You must run your code first

Time and Space Complexity Analysis

Time Complexity

- **Building the Trie**: (O(N x L)), where (N) is the number of products and (L) is the average length of the products.
- **Searching for Each Prefix**: (O(M x K)), where (M) is the length of the search word and (K) is the time taken for the DFS, which is limited to 3 (constant time). Overall, it's approximately (O(M)).

Overall Time Complexity: O(N * L + M), combining the time to build the Trie and perform searches.

Space Complexity

- **Trie Storage**: (O(N x L)), as each product of average length (L) is stored in the trie.
- **Search Results**: (O(M)), as we store up to 3 suggestions for each character in the search word.

Overall, the space complexity is dominated by the trie storage, which is (O(N x L)).

← Previous
Search Suggestions System (Medium)

Next →
Introduction To Topological Sort Pattern

☐ Mark as Completed